

Experiment No. 1

Title

8-Puzzle Problem Using Artificial Intelligence

Aim

To develop a Python program that solves the 8-Puzzle problem using the **A*** (A-Star) search algorithm.

Objective

The objective of this experiment is to understand state-space search and implement the A* search algorithm to find the shortest sequence of moves from the initial state to the goal state in the 8-Puzzle problem.

Algorithm

Step 1: Define the initial state and the goal state.

Step 2: Calculate the heuristic value (Manhattan Distance) for the current state.

Step 3: Insert the initial state into the priority queue.

Step 4: Remove the state having the lowest cost.

Step 5: Check whether the current state is the goal state.

- If yes, display the solution and stop.

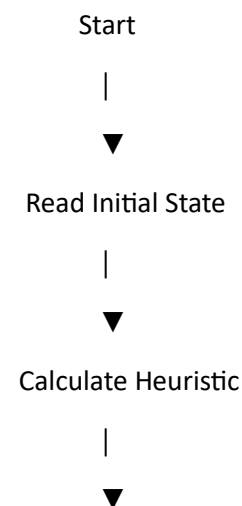
Step 6: Generate all possible next states by moving the blank tile.

Step 7: Compute the cost for each successor state.

Step 8: Insert unexplored states into the priority queue.

Step 9: Repeat Steps 4–8 until the goal state is reached.

Flowchart



Insert into Priority Queue

|



Remove Lowest Cost Node

|



Goal State Reached?

|

Yes

|

No

|



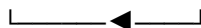
|



Display Path Generate Children

|

|



|



Stop

Python Program

```
from queue import PriorityQueue
```

```
goal = [[1,2,3],
```

```
        [4,5,6],
```

```
        [7,8,0]]
```

```
def heuristic(state):
```

```
    distance = 0
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            value = state[i][j]
```

```
            if value != 0:
```

```
    goal_x = (value - 1) // 3
    goal_y = (value - 1) % 3

    distance += abs(i - goal_x) + abs(j - goal_y)

return distance
```

```
def get_neighbors(state):
```

```
    neighbors = []
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j
```

```
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
    for dx, dy in moves:
```

```
        nx, ny = x + dx, y + dy
```

```
        if 0 <= nx < 3 and 0 <= ny < 3:
```

```
            new_state = [row[:] for row in state]
```

```
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
```

```
            neighbors.append(new_state)
```

```
    return neighbors
```

```
def solve(start):
```

```
    pq = PriorityQueue()
```

```
    pq.put((heuristic(start), 0, start, []))
```

```
    visited = set()
```

```
    while not pq.empty():
```

```
        f, g, state, path = pq.get()
```

```
state_tuple = tuple(map(tuple, state))
```

```
if state_tuple in visited:
```

```
    continue
```

```
visited.add(state_tuple)
```

```
if state == goal:
```

```
    return path + [state]
```

```
for neighbor in get_neighbors(state):
```

```
    pq.put((g + 1 + heuristic(neighbor),
```

```
           g + 1,
```

```
           neighbor,
```

```
           path + [state]))
```

```
return None
```

```
start = [
```

```
    [1,2,3],
```

```
    [4,0,6],
```

```
    [7,5,8]
```

```
]
```

```
solution = solve(start)
```

```
if solution:
```

```
    print("Solution Found:\n")
```

```
    for step in solution:
```

```
        for row in step:
```

```
            print(row)
```

```
        print()
```

```
else:
```

```
print("No Solution")
```

Sample Output

Solution Found:

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Result

The 8-Puzzle problem was successfully solved using the **A*** search algorithm. The shortest sequence of moves from the initial state to the goal state was obtained.

Conclusion

Thus, the A* search algorithm efficiently solved the 8-Puzzle problem by using the Manhattan Distance heuristic to find the optimal path with minimum cost.

-S